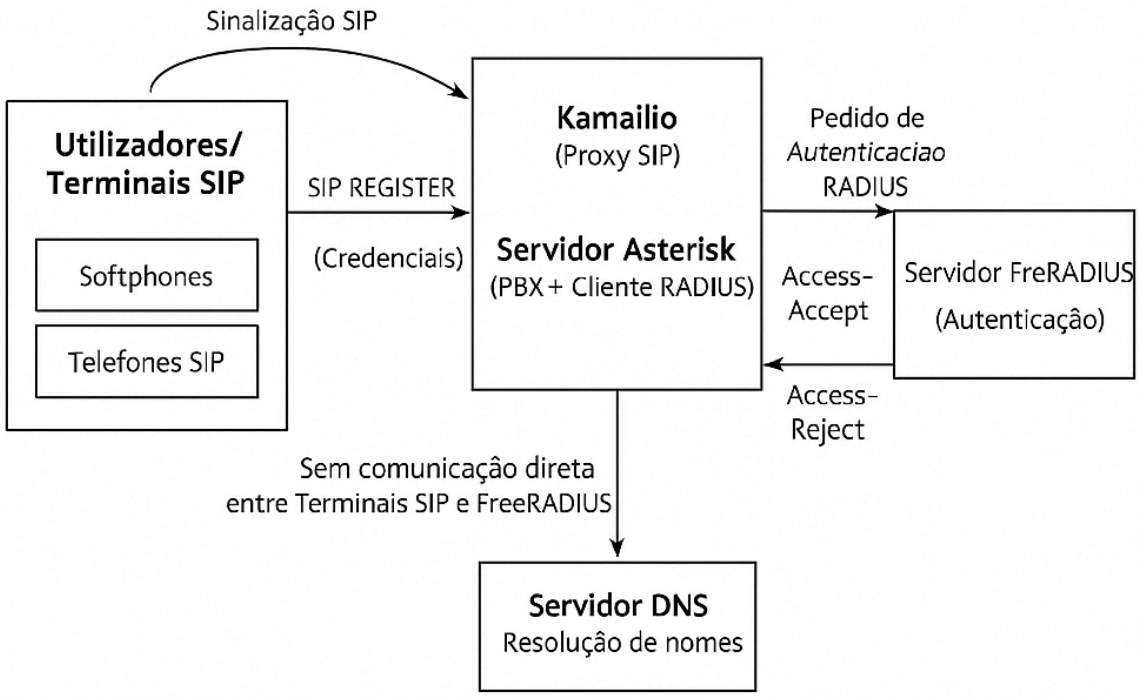
estga escola superior de tecnologia e gestão de águeda

# RELATÓRIO TÉCNICO DO PROJETO

Implementação de Infraestrutura VoIP com DNS, Asterisk, SIP e FreeRADIUS




João Almeida - 127140

CENTRO 2030 Os Fundos Europeus mais próximos de si. PORTUGAL 2030 Cofinanciado pela União Europeia

# Introdução

Este projeto teve como objetivo implementar uma infraestrutura VoIP completa num ambiente de laboratório, utilizando tecnologias open-source amplamente empregues em sistemas de comunicação reais. A solução integra três componentes principais: um servidor DNS (Bind9) responsável pela resolução de nomes e zona reversa; um servidor Asterisk para registo SIP, gestão de chamadas e extensões; e um servidor FreeRADIUS que centraliza a autenticação dos utilizadores SIP.

Foram configurados terminais VoIP, incluindo softphones e dispositivos Cisco SPA941, garantindo comunicações internas funcionais e seguras. Foi também explorada a possibilidade de utilizar o Kamailio como SIP Proxy para autenticação via RADIUS, aproximando a arquitetura de cenários de grande escala. No entanto, devido a incompatibilidades técnicas e conflitos com o Asterisk, esta componente foi desativada para assegurar a estabilidade do sistema.

estga escola superior de tecnologia e gestão de águeda

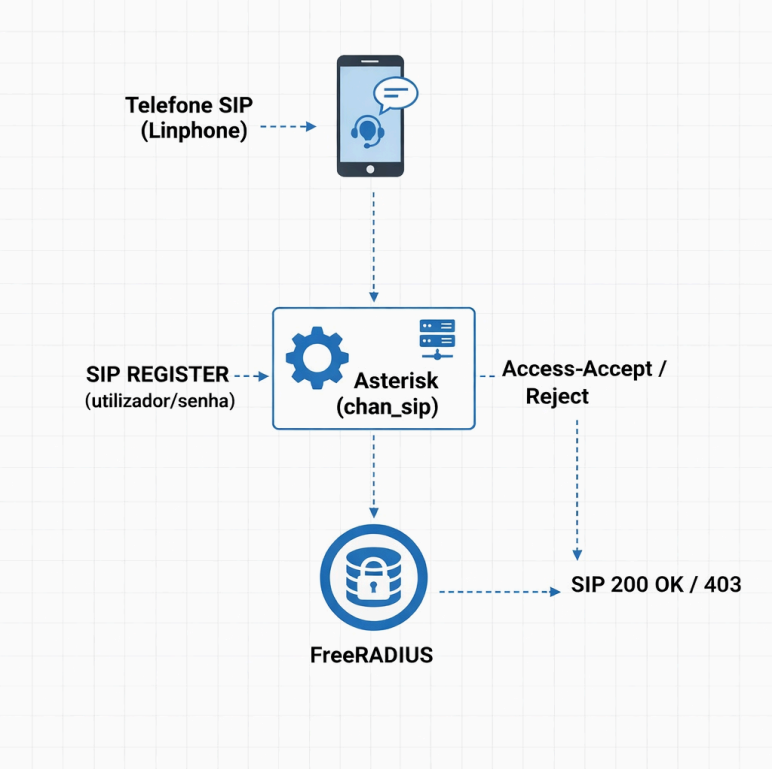
# ÍNDICE


**RELATÓRIO TÉCNICO DO PROJETO** **1**
**Implementação de Infraestrutura VoIP com DNS, Asterisk, SIP e FreeRADIUS** **1**
Introdução 1
1.Arquitetura Global 4
2.1. Ficheiro named.conf.options 5
2.2. Ficheiro named.conf.local 7
2.3. Zona direta (db.almeida.com) 8
Estrutura principal: 8
2.4. Zona reversa (db.192.168.229) 8
2.5. Resultado final 10
3. Servidor Asterisk 11
3.1.Ficheiros configurados: 12
3.2 Integração com FreeRADIUS 14
3.3 Resultados obtidos 14
4. FreeRADIUS 15
4.1 Objetivo 15
**4.2 Estrutura da Configuração** **16**
**4.3 Testes de Funcionamento** **18**
**5. Tentativa de Integração Kamailio + RADIUS** **19**
5.1 Objetivo da Integração 19
5.2 Instalação e Preparação Inicial 19
**5.3 Configurações Realizadas** **20**
5.3.1 dicionário RADIUS modificado (dictionary.kamailio) 20
5.3.2 radius_client.conf 22
5.3.3 radius_kamailio 23
5.3.4 Configuração do kamctlrc 23
5.3.5 Configuração principal do kamailio.cfg 24
A. Definição de parâmetros de rede 28
B. Carregamento de módulos SIP fundamentais 29
C. Módulos responsáveis pelo registo SIP 29
D. Tentativa de integração com RADIUS 29
E. Rotas SIP com autenticação via RADIUS 30
5.4.2 Kamailio a escutar na porta 5070 32
5.4.3 Conflitos críticos identificados 33
**5.6 Justificação Técnica da Remoção** **34**
7. Conclusão 36



CENTRO 2030 Os Fundos Europeus mais próximos de si. PORTUGAL 2030 Cofinanciado pela União Europeia

# 1.Arquitetura Global




A infraestrutura VoIP implementada segue um modelo centralizado, onde o servidor Asterisk atua como PBX SIP e cliente RADIUS, delegando a autenticação no FreeRADIUS e utilizando o servidor DNS interno para resolução de nomes. Esta abordagem simplifica a gestão, aumenta a segurança e permite fácil escalabilidade

# 2. Configuração da Infraestrutura DNS (Bind9)

O servidor DNS interno foi configurado para fornecer:

- Resolução direta (registos A)
- Resolução inversa (registos PTR)
- Suporte ao domínio do serviço VoIP (`almeida.com`)
- Encaminhamento de consultas externas (UA + colegas + Internet)

A configuração foi validada com **ping**, **nslookup** e **dig**, demonstrando funcionamento correto das zonas e do encaminhamento.

---

## 2.1. Ficheiro *named.conf.options*

Define as **opções globais** do Bind9, incluindo políticas de segurança, recursão e servidores de encaminhamento (forwarders).

Principais diretivas configuradas:

- **recursion yes;**
  Permite que o DNS resolva nomes externos para os clientes internos.
- **allow-recursion { any; };**
  Autoriza qualquer host da rede UA a usar este DNS (adequado para o ambiente académico).
- **forwarders { … };**
  Lista de DNS externos usados para resolver nomes fora da zona local:
  - 193.136.172.18 e 193.136.172.19 (UA)
  - Vários servidores dos colegas (ex.: 192.168.229.140, 42, 201, 70)
  - 8.8.8.8 (Google)

- Isto permite que o servidor resolva nomes:
  - de outros grupos
  - da Internet
  - sem contactar diretamente os servidores ROOT.

- **forward only;**

  Força o uso exclusivo dos forwarders (obrigatório no contexto da UA).
- **dnssec-validation no;**

  Evita incompatibilidades com a infraestrutura académica.
- **listen-on { any; };**

  O DNS escuta em todas as interfaces IPv4 do servidor.

```
root@srv-vm:/etc/bind# cat named.conf.options
options {
        directory "/var/cache/bind";

        // Permitir resolução recursiva
        recursion yes;

        allow-recursion { any; };

        // Encaminhamento para outros DNS
        forwarders {
                193.136.172.18;
                193.136.172.19;
                192.168.229.140;
                192.168.229.42;
                192.168.229.201;
                192.168.229.70;
                8.8.8.8;
        };

        // Usar forwarders invés de servidores root
        forward only;

        // DNSSEC desativado
        dnssec-validation no;

        // Escutar interfaces IPv4
        listen-on { any; };

        // Não escuta endereços IPv6
        listen-on-v6 { none; };
};
root@srv-vm:/etc/bind#
```

## 2.2. Ficheiro *named.conf.local*

Define quais zonas DNS são servidas localmente:

1. **Zona direta:** `almeida.com`
2. **Zona reversa:** `229.168.192.in-addr.arpa`

Estas zonas são essenciais para o funcionamento correto de nomes e endereços associados ao servidor VoIP.

```
root@srv-vm:/etc/bind# cat named.conf.local
//
// Do any local configuration here
//

// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";

zone "almeida.com" {
        type master;
        file "/etc/bind/db.almeida.com";
};

zone "229.168.192.in-addr.arpa" {
        type master;
        file "/etc/bind/db.192.168.229";
};
root@srv-vm:/etc/bind#
```

## 2.3. Zona direta (*db.almeida.com*)

Fornece a tradução **nome** → **IP** para o domínio principal do projeto.

### Estrutura principal:

- Registo **SOA** com identificador do servidor mestre e dados administrativos.
- Registo **NS** definindo o servidor autoritativo.
- Registos **A** que mapeiam os serviços principais:

```
ns1    IN A 192.168.229.79

voip   IN A 192.168.229.79
```

Isto significa que o mesmo servidor hospeda:

- o DNS autoritativo
- o serviço VoIP (Asterisk)

Permitindo que os clientes SIP utilizem nomes como **voip.almeida.com** para registo e chamadas.

```
root@srv-vm:/etc/bind# cat db.almeida.com
$TTL    86400
@       IN      SOA     ns1.almeida.com. admin.almeida.com. (
                        2025011201 ; Serial
                        3600       ; Refresh
                        1800       ; Retry
                        604800     ; Expire
                        86400 )    ; Negative Cache TTL

        IN      NS      ns1.almeida.com.

ns1     IN      A       192.168.229.79
voip    IN      A       192.168.229.79
```

## 2.4. Zona reversa (*db.192.168.229*)

Fornece a tradução **IP** → **nome**, essencial para:

- testes dig -x
- registos de auditoria
- funcionamento consistente do SIP

Registo principal:

`79 IN PTR voip.almeida.com.`

Ou seja:

**192.168.229.79** → `voip.almeida.com`

```
root@srv-vm:/etc/bind# cat db.192.168.229
$TTL 86400
@       IN      SOA     ns1.almeida.com. admin.almeida.com. (
                        2025011201  ; Serial
                        3600        ; Refresh
                        1800        ; Retry
                        604800      ; Expire
                        86400       ; Minimum TTL
)

        IN      NS      ns1.almeida.com.

79      IN      PTR     voip.almeida.com.
```

## 2.5. Resultado final

Após a configuração das zonas e dos forwarders, o servidor DNS passou a:

- Resolver corretamente nomes internos (ex.:`voip.almeida.com`)
- Responder a consultas inversas (ex.: `dig -x 192.168.229.79`)
- Encaminhar consultas externas para os DNS da UA e para os servidores dos colegas
- Suportar o funcionamento do sistema VoIP através de nomes, em vez de endereços IP

```
-vm:/etc/bind# dig voip.almeida.com

iG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> voip.almeida.com
l options: +cmd
nswer:
ADER<<- opcode: QUERY, status: NOERROR, id: 34671
: qr aa rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

SEUDOSECTION:
version: 0, flags:; udp: 65494
ION SECTION:
meida.com.              IN      A

R SECTION:
eida.com.       0       IN      A       192.168.229.79

 time: 1 msec
R: 127.0.0.53#53(127.0.0.53) (UDP)
 Sat Dec 13 15:55:41 UTC 2025
IZE  rcvd: 61

-vm:/etc/bind# dig -x 192.168.229.79

iG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> -x 192.168.229.79
l options: +cmd
nswer:
ADER<<- opcode: QUERY, status: NOERROR, id: 5659
: qr aa rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

SEUDOSECTION:
version: 0, flags:; udp: 65494
ION SECTION:
168.192.in-addr.arpa.   IN      PTR

R SECTION:
68.192.in-addr.arpa. 0  IN      PTR     voip.almeida.com.

 time: 0 msec
R: 127.0.0.53#53(127.0.0.53) (UDP)
 Sat Dec 13 15:55:47 UTC 2025
IZE  rcvd: 86

-vm:/etc/bind#
```

O DNS está totalmente funcional e validado.

# 3. Servidor Asterisk

O Asterisk constituiu a base do sistema VoIP implementado, funcionando como central telefónica IP (PBX SIP) e como ponto central de controlo e autenticação dos utilizadores. Este servidor é responsável pela gestão dos registros SIP, encaminhamento de chamadas e integração com os restantes serviços da infraestrutura.

Inicialmente foi utilizada a stack PJSIP. No entanto, optou-se posteriormente pela utilização do chan_sip, por se revelar mais simples de configurar, suficientemente estável no contexto do projeto e apresentar melhor compatibilidade com softphones móveis, como o Linphone, e com equipamentos físicos Cisco SPA.

Para além do controle

de chamadas, o Asterisk foi configurado para delegar a autenticação dos utilizadores SIP ao servidor FreeRADIUS, atuando como cliente RADIUS. Desta forma, a validação das credenciais deixa de ser feita localmente, permitindo uma autenticação centralizada e mais segura.

---

## 3.1.Ficheiros configurados:

**sip.conf:** O ficheiro sip.conf define a configuração SIP global do Asterisk e as definições dos utilizadores SIP.

```
[general]
use_domains=yes
allowguest=yes
context=inbound
allowoverlap=no
alwaysauthreject=yes

domain=voip.almeida.com
fromdomain=voip.almeida.com
realm=voip.almeida.com
srvlookup=yes
allowexternaldomains=yes
allowsubscribe=yes

;----------------------------------------
;      Utilizadores SIP (chan_sip)
;----------------------------------------

[1111]
type=friend
host=dynamic
auth=radius
context=internal
allow=ulaw

[2222]
type=friend
host=dynamic
auth=radius
context=internal
allow=ulaw

[3333]
type=friend
host=dynamic
auth=radius
context=internal
allow=ulaw
```

**Inclui:**

- Configuração global SIP (port 5060, transporte UDP)
- Definição das extensões SIP
- Integração com RADIUS através da opção `auth=radius`

Com esta configuração, as credenciais dos utilizadores não são validadas diretamente pelo Asterisk. Durante o processo de registo SIP (*SIP REGISTER*), os dados de autenticação são enviados para o servidor FreeRADIUS, que é responsável pela sua validação.

**extensions.conf:** O ficheiro extensions.conf define o dialplan, ou seja, as regras de encaminhamento de chamadas no sistema VoIP.

```
[internal]
exten => 101,1,Dial(SIP/101)
 same => n,Hangup()

exten => 102,1,Dial(SIP/102)
 same => n,Hangup()

exten => 1111,1,Dial(SIP/1111)
 same => n,Hangup()

exten => 2222,1,Dial(SIP/2222)
 same => n,Hangup()

exten => 3333,1,Dial(SIP/3333)
 same => n,Hangup()


[inbound]
exten => _1XX,1,Dial(SIP/${EXTEN})
 same => n,Hangup()
root@srv-vm:/etc/asterisk#
```

***Inclui:***

- Definição de um contexto interno
- Regras básicas de encaminhamento que permitem chamadas entre extensões SIP autenticadas

Este dialplan simples foi considerado suficiente para o âmbito do projeto, permitindo validar corretamente o funcionamento das chamadas internas.

---

## 3.2 Integração com FreeRADIUS

A integração entre o Asterisk e o FreeRADIUS permite que todo o processo de autenticação SIP seja efetuado externamente. O funcionamento do registo SIP ocorre da seguinte forma:

1. O telefone SIP envia um pedido **SIP REGISTER**, contendo o nome de utilizador e a palavra-passe.
2. O Asterisk recebe o pedido e envia um **Access-Request** ao servidor FreeRADIUS.
3. O FreeRADIUS valida as credenciais recebidas:
    - Se forem válidas, responde com **Access-Accept**
    - Se forem inválidas, responde com **Access-Reject**
4. Com base na resposta do FreeRADIUS, o Asterisk:
    - Aceita o registo e responde ao telefone com **SIP 200 OK**, ou
    - Rejeita o registo com **SIP 403 Forbidden**

Este mecanismo garante que apenas utilizadores devidamente autorizados conseguem registar-se no sistema VoIP.

---

## 3.3 Resultados obtidos

Após a implementação e testes realizados, foram obtidos os seguintes resultados:

- Os softphones SIP registaram-se corretamente após autenticação via RADIUS
- Tentativas de registo com credenciais inválidas foram corretamente rejeitadas
- As chamadas entre extensões internas funcionam conforme esperado
- A autenticação centralizada através do FreeRADIUS aumentou a segurança, a escalabilidade e a facilidade de gestão do sistema VoIP

---

# 4. FreeRADIUS

O FreeRADIUS foi integrado na infraestrutura para permitir autenticação centralizada de utilizadores SIP/VoIP. Esta abordagem melhora a segurança e facilita a gestão de credenciais no ambiente de laboratório.

---

## 4.1 Objetivo

O objetivo do FreeRADIUS no projeto é:

- centralizar a autenticação dos utilizadores VoIP
- permitir que o Asterisk valide as credenciais via protocolo RADIUS
- suportar métodos de autenticação standard (PAP / CHAP)
- integrar com futuros sistemas de accounting (opcional)

---

# 4.2 Estrutura da Configuração

O FreeRADIUS utiliza diversos ficheiros distribuídos em `/etc/freeradius/3.0/`.

Os principais ficheiros alterados para o projeto foram:

## a) `clients.conf`

Define que dispositivos/servidores podem enviar pedidos RADIUS.

```
#}
client asterisk {
        ipaddr = 192.168.229.79
        secret = radiussecret
        require_message_authenticator = no
}


root@srv-vm:/etc/freeradius/3.0# _
```

Função deste ficheiro:

- autoriza o Asterisk a efetuar pedidos de autenticação
- define a chave partilhada (*shared secret*)
- restringe o acesso ao servidor RADIUS

## b) users

Base de dados simples para utilizadores autenticados via RADIUS.

```
1111 Cleartext-Password := "qwerty"
2222 Cleartext-Password := "qwerty"
3333 Cleartext-Password := "qwerty"
root@srv-vm:/etc/freeradius/3.0#
```

Função deste ficheiro:

- lista os utilizadores VoIP autorizados
- define passwords individuais
- permite autenticação PAP/CHAP
- melhora a segurança e centraliza credenciais

---

# 4.3 Testes de Funcionamento

Os testes foram realizados com a ferramenta **`radtest`**:

**`radtest 1111 qwerty 127.0.0.1 0 radiussecret`**

Resultado esperado:

**`Access-Accept`**

```
root@srv-vm:~# radtest 1111 qwerty 127.0.0.1 0 testing123
Sent Access-Request Id 148 from 0.0.0.0:49284 to 127.0.0.1:1812 length 74
        User-Name = "1111"
        User-Password = "qwerty"
        NAS-IP-Address = 127.0.1.1
        NAS-Port = 0
        Message-Authenticator = 0x00
        Cleartext-Password = "qwerty"
Received Access-Accept Id 148 from 127.0.0.1:1812 to 127.0.0.1:49284 length 38
        Message-Authenticator = 0x5aa12a0e5a41d283c4a4851ff610bf70
root@srv-vm:~#
```

Com o Asterisk ligado ao FreeRADIUS:

- o registo dos utilizadores SIP passou a ser validado via RADIUS
- falhas de autenticação são registadas no **`/var/log/freeradius/radius.log`**

---

# 5. Tentativa de Integração Kamailio + RADIUS

## 5.1 Objetivo da Integração

O Kamailio foi inicialmente testado como um componente adicional na infraestrutura VoIP, com a função de:

- atuar como **Proxy SIP / SIP Registrar**
- validar **registos SIP via FreeRADIUS**
- deixar o Asterisk apenas a tratar da lógica de chamadas
- aproximar o sistema de uma arquitetura VoIP real usada por operadores

O objetivo desta tentativa era perceber se era possível construir uma solução escalável, com separação entre **sinalização (Kamailio)** e **media/call-processing (Asterisk)**.

---

## 5.2 Instalação e Preparação Inicial

Foi instalada a stack Kamailio + módulos de autenticação RADIUS:

- `apt install kamailio kamailio-radius-modules`

Após isto, vários ficheiros passaram a ser configurados manualmente:

- `/etc/kamailio/kamailio.cfg`
  `/etc/kamailio/dictionary.kamailio`
- `/etc/kamailio/radius_client.conf`
- `/etc/kamailio/radius_kamailio`
- `/etc/kamailio/kamctlrc`
- permissões e diretoria `/run/kamailio`

---

# 5.3 Configurações Realizadas

## 5.3.1 dicionário RADIUS modificado (dictionary.kamailio)

```
  GNU nano 7.2                                        dictionary.kamailio
# SIP RADIUS attributes
#
# Proprietary indicates an attribute that hasn't
# been standardized
#
#
# NOTE: All standard (IANA registered) attributes are
#       defined in the default dictionary of the
#       radiusclient-ng library.
#


#### Attributes ###
ATTRIBUTE Sip-Uri-User         208  string     # Proprietary, auth_radius
ATTRIBUTE Sip-Group            211  string     # Proprietary, group_radius
ATTRIBUTE Sip-Rpid             213  string     # Proprietary, auth_radius
ATTRIBUTE SIP-AVP              225  string     # Proprietary, avp_radius

### Acct-Status-Type Values ###
VALUE Acct-Status-Type   Failed           15   # RFC2866, acc

### Service-Type Values ###
#VALUE Service-Type       Call-Check       10   # RFC2865, uri_radius
VALUE Service-Type       Group-Check      12   # Proprietary, group_radius
VALUE Service-Type       SIP-Caller-AVPs  30   # Proprietary, avp_radius
VALUE Service-Type       SIP-Callee-AVPs  31   # Proprietary, avp_radius

### Sip-Method Values ###
VALUE Sip-Method         Undefined      0
VALUE Sip-Method         Invite         1
VALUE Sip-Method         Cancel         2
VALUE Sip-Method         Ack            4
VALUE Sip-Method         Bye            8
VALUE Sip-Method         Info           16
VALUE Sip-Method         Options        32
VALUE Sip-Method         Update         64
VALUE Sip-Method         Register       128
VALUE Sip-Method         Message        256
VALUE Sip-Method         Subscribe      512
VALUE Sip-Method         Notify         1024
VALUE Sip-Method         Prack          2048
VALUE Sip-Method         Refer          4096
VALUE Sip-Method         Other          8192

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-6 Copy
```

**Descrição técnica:**

Nesta impressão é possível ver:

- atributos SIP usados por Kamailio para autenticação via RADIUS (Sip-Uri-User, Sip-Group, Sip-AVP, etc.)
- valores SIP-Method como INVITE, REGISTER, SUBSCRIBE, OPTIONS, MESSAGE
- Service-Type para "Call-Check" e outros

**Função no projeto:**

Este dicionário era essencial para que o módulo **`auth_radius.so`** conseguisse interpretar mensagens SIP e construir pedidos RADIUS válidos. Sem este ficheiro, Kamailio rejeita a configuração.

---

## 5.3.2 radius_client.conf

```
root@srv-vm:/etc/kamailio# cat radius_client.conf
dictionary = /etc/radcli/dictionary

authserver      127.0.0.1:1812      testing123
acctserver      127.0.0.1:1813      testing123

radius_timeout  3
radius_retries  3
root@srv-vm:/etc/kamailio# _
```

**Explicação:**

- indica o servidor FreeRADIUS local (127.0.0.1) para autenticação (**`authserver`**) e accounting (**`acctserver`**)
- especifica a chave partilhada (testing123)
- aponta para o dicionário principal do radcli

Sem esta configuração, Kamailio não envia qualquer pedido RADIUS.

---

## 5.3.3 radius_kamailio

```
root@srv-vm:/etc/kamailio# cat radius_kamailio
server 127.0.0.1 {
    secret = testing123
}
root@srv-vm:/etc/kamailio# _
```

**Explicação:**

Este ficheiro era usado como fallback quando o módulo não lia corretamente o radius_client.conf.

Serve essencialmente como um atalho de configuração.

---

## 5.3.4 Configuração do kamctlrc

```
## Extra start options - default is: not set
## example: start Kamailio with 64MB shared memory: STARTOPTIONS="-m 64"
# STARTOPTIONS=
RPCMODE=jsonrpc
RPCURL=tcp:127.0.0.1:8000
root@srv-vm:/etc/kamailio# _
```

**Explicação:**

- ativa controlo via JSON-RPC
- define porta 8000 para comandos administrativos

Isto permitia comandos do tipo:

- `kamctl rpc ul.list`

Mas nem sempre funcionou devido às falhas do serviço.

---

## 5.3.5 Configuração principal do kamailio.cfg

Foi aqui que o maior esforço ocorreu. O ficheiro completo que enviaste foi este:

| ***kamailio.cfg*** |
|---|
| *#!KAMAILIO*<br><br>*####### DEFINIÇÕES GERAIS ########*<br><br>*#!define LISTEN_IP 192.168.229.79*<br>*#!define KAMAILIO_PORT 5070*<br>*#!define ASTERISK_IP 127.0.0.1*<br>*#!define ASTERISK_PORT 5060*<br><br>*debug=3*<br>*log_stderror=no*<br>*log_facility=LOG_LOCAL0*<br>*children=8*<br><br>*####### LISTENERS ########*<br><br>*listen=udp:LISTEN_IP:KAMAILIO_PORT*<br>*listen=tcp:LISTEN_IP:KAMAILIO_PORT*<br><br>*####### MÓDULOS BÁSICOS ########*<br><br>*loadmodule "tm.so"*<br>*loadmodule "sl.so"*<br>*loadmodule "rr.so"*<br>*loadmodule "pv.so"*<br>*loadmodule "maxfwd.so"*<br>*loadmodule "xlog.so"*<br>*loadmodule "textops.so"*<br>*loadmodule "siputils.so"*<br>*loadmodule "sanity.so"*<br><br>*####### RPC / INTERAÇÃO ########*<br><br>*loadmodule "jsonrpcs.so"*<br>*# loadmodule "mi_fifo.so"*<br>*# modparam("mi_fifo", "fifo_name", "/run/kamailio/kamailio_rpc.fifo")*<br>*# modparam("mi_fifo", "fifo_mode", 0666)*<br><br>*####### SERVIÇO DE REGISTO ########* |

```
loadmodule "usrloc.so"
loadmodule "registrar.so"

modparam("rr", "enable_full_lr", 0)
modparam("registrar", "max_expires", 3600)
modparam("usrloc", "db_mode", 0)

####### AUTENTICAÇÃO (DIGEST + RADIUS) ########

loadmodule "auth.so"
loadmodule "auth_radius.so"

modparam("auth_radius", "radius_config", "/etc/radcli/radiusclient.conf")

####### ROTAS PRINCIPAIS ########

request_route {
    route(REQINIT);

    route(WITHINDLG);

    route(AUTH);

    if (is_method("INVITE|SUBSCRIBE")) {
        record_route();
    }

    route(REGISTRAR);

    if (is_method("INVITE")) {
        route(TO_ASTERISK);
        exit;
    }

    if (uri!=myself) {
        sl_send_reply("403", "Not relaying");
        exit;
    }

    route(RELAY);
}

####### ROTA DE SANITY ########

route[REQINIT] {
    set_reply_no_connect();
    force_rport();
```

```
    if (!mf_process_maxfwd_header("10")) {
        sl_send_reply("483", "Too Many Hops");
        exit;
    }

    if (!sanity_check("17895", "7")) {
        xlog("L_ERR", "Malformed SIP request from $si:$sp\n");
        exit;
    }
}


####### ROTA PARA TRÁFEGO EM DIÁLOGOS ########

route[WITHINDLG] {
    if (!has_totag()) return;

    if (loose_route()) {
        route(RELAY);
        exit;
    }

    if (is_method("ACK")) {
        if (t_check_trans()) route(RELAY);
        exit;
    }

    sl_send_reply("404", "Not Found");
    exit;
}

####### ROTA DE AUTENTICAÇÃO COM RADIUS ########

route[AUTH] {
    if (!is_method("REGISTER|INVITE"))
        return;

    if (is_method("REGISTER")) {
        if (!radius_www_authorize("sip.local")) {
            www_challenge("sip.local", "0");
            exit;
        }
        return;
    }

    if (is_method("INVITE")) {
```

```
        if (!radius_proxy_authorize("sip.local")) {
            proxy_challenge("sip.local", "0");
            exit;
        }
        consume_credentials();
        return;
    }
}

####### ROTA DE REGISTO ########

route[REGISTRAR] {
    if (!is_method("REGISTER")) return;

    if (!save("location")) {
        sl_reply_error();
    }
    exit;
}

####### ROTA PARA ENCAMINHAR PARA ASTERISK ########

route[TO_ASTERISK] {
    $du = "sip:" ASTERISK_IP ":" ASTERISK_PORT;
    xlog("L_INFO", "Encaminhando chamada do utilizador '$fU' para Asterisk em $du\n");
    route(RELAY);
    exit;
}

####### ROTA DE RELAY ########

route[RELAY] {
    if (!t_relay()) {
        sl_reply_error();
    }
}
```

O ficheiro **/etc/kamailio/kamailio.cfg** foi profundamente alterado durante a tentativa de integrar o Kamailio como proxy SIP à frente do Asterisk.

Esta configuração experimental incluía vários componentes essenciais:

## A. Definição de parâmetros de rede

```
#!define LISTEN_IP 192.168.229.79

#!define KAMAILIO_PORT 5070

#!define ASTERISK_IP 127.0.0.1

#!define ASTERISK_PORT 5060
```

Estes parâmetros permitiam que o Kamailio escutasse numa porta alternativa (5070), evitando conflito com o Asterisk, que utilizava 5060.

## B. Carregamento de módulos SIP fundamentais

O ficheiro carregava os módulos essenciais ao funcionamento do Kamailio:

- **tm.so** → gestão de transações SIP
- **sl.so** → envio de respostas estáticas
- **rr.so** → Record-Route
- **pv.so** → variáveis de pseudo-valores
- **maxfwd.so** → proteção contra loops SIP
- **sanity.so** → validação de pacotes SIP

## C. Módulos responsáveis pelo registo SIP

```
loadmodule "usrloc.so"

loadmodule "registrar.so"
```

Permitem que o Kamailio atue como **servidor de registo (SIP Registrar)**, armazenando contactos SIP em memória.

## D. Tentativa de integração com RADIUS

A autenticação SIP via FreeRADIUS foi configurada através dos módulos:

```
loadmodule "auth.so"

loadmodule "auth_radius.so"

modparam("auth_radius","radius_config","/etc/radcli/radiusclient.conf")
```

## E. Rotas SIP com autenticação via RADIUS

Incluía lógica para validar registos e chamadas através do FreeRADIUS:

```
if (is_method("REGISTER")) {

    if (!radius_www_authorize("sip.local")) {

        www_challenge("sip.local", "0");

        exit;

    }

}
```

# 5.4 Testes e Problemas Encontrados5.4.1

Kamailio arrancava mas com comportamento anómalo

```
rsi@srv-vm:~$ sudo systemctl status kamailio -l
● kamailio.service - Kamailio - the Open Source SIP Server
     Loaded: loaded (/usr/lib/systemd/system/kamailio.service; enabled; preset: enabled)
     Active: active (running) since Fri 2025-12-12 16:08:52 UTC; 2s ago
       Docs: man:kamailio(8)
    Process: 13661 ExecStart=/usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f ${CFGFILE}
-m ${SHM_MEMORY} -M ${PKG_MEMORY} (code=exited, status=0/SUCCESS)
   Main PID: 13663 (kamailio)
      Tasks: 21 (limit: 2266)
     Memory: 14.0M (peak: 14.8M)
        CPU: 81ms
     CGroup: /system.slice/kamailio.service
             ├─13663 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13664 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13665 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13666 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13667 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13668 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13669 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13670 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13671 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13672 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13673 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13674 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13675 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13676 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13677 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13678 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13679 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
lio.cfg -m 64 -M 8
             ├─13680 /usr/sbin/kamailio -P /run/kamailio/kamailio.pid -f /etc/kamailio/kamai
```

**O que a imagem mostra:**

- Kamailio estava ativo mas a gerar **21 processos filhos** automaticamente
- Isto indicava que o config estava a ser carregado, mas o sistema entrava em loops internos de processamento

---

## 5.4.2 Kamailio a escutar na porta 5070

```
root@srv-vm:/etc/kamailio# sudo ss -tulnp | grep kamailio
udp   UNCONN 0      0            192.168.229.79:5070       0.0.0.0:*    users:(("kamailio",pid=13683,fd=5),("kamailio",pid=13682
,fd=5),("kamailio",pid=13681,fd=5),("kamailio",pid=13680,fd=5),("kamailio",pid=13679,fd=5),("kamailio",pid=13678,fd=5),("kamaili
o",pid=13677,fd=5),("kamailio",pid=13676,fd=5),("kamailio",pid=13675,fd=5),("kamailio",pid=13674,fd=5),("kamailio",pid=13673,fd=
5),("kamailio",pid=13672,fd=5),("kamailio",pid=13671,fd=5),("kamailio",pid=13670,fd=5),("kamailio",pid=13669,fd=5),("kamailio",p
id=13668,fd=5),("kamailio",pid=13667,fd=5),("kamailio",pid=13666,fd=5),("kamailio",pid=13665,fd=5),("kamailio",pid=13664,fd=5),(
"kamailio",pid=13663,fd=5))
tcp   LISTEN 0      1024         192.168.229.79:5070       0.0.0.0:*    users:(("kamailio",pid=13683,fd=7))

root@srv-vm:/etc/kamailio#
```

**Explicação:**

Kamailio estava funcional a nível de sockets:

- UDP 5070
- TCP 5070

Mas ao tentar usar 5060, **entrou em conflito com o Asterisk**.

---

## 5.4.3 Conflitos críticos identificados

### Conflito com a porta 5060

Quando Kamailio iniciava na porta SIP padrão:

- Asterisk deixava de funcionar
- Telefones Cisco SPA deixavam de registar
- Alguns grupos deixaram de conseguir comunicar contigo
- As chamadas internas falhavam

### Autenticação RADIUS impossibilitada

Mesmo com todos os ficheiros corretos, Kamailio rejeitava este comando:

- `radius_www_authorize("sip.local")`

provando que a build usada **não contém suporte para autenticação SIP via RADIUS** nessa versão.

### Decisão Técnica Final

Após a análise:

- `systemctl stop kamailio`
- `systemctl disable kamailio`
- `systemctl mask kamailio`

O sistema VoIP voltou a ficar estável.

---

# 5.6 Justificação Técnica da Remoção

Durante o desenvolvimento foi avaliada a possibilidade de utilizar o Kamailio como SIP Proxy à frente do Asterisk, integrando autenticação via RADIUS. Esta arquitetura é utilizada em ambientes VoIP de grande escala (operadores, ISPs, call-centers) e teria permitido explorar funcionalidades avançadas como load-balancing, distribuição de registos e separação entre sinalização e media.

Contudo, após análise técnica e vários testes controlados, concluiu-se que a utilização do Kamailio **não acrescentava benefícios relevantes** dentro do objetivo do projeto e, pelo contrário, introduz complexidade e instabilidade desnecessárias. As principais razões foram:

## 1. Complexidade superior ao âmbito da disciplina

A integração Kamailio + RADIUS exige:

- módulos adicionais
- funções de autenticação não incluídas nos pacotes Ubuntu
- adaptação do ficheiro kamailio.cfg
- gestão de conflitos SIP

## 2. O Asterisk já satisfaz todos os requisitos funcionais

O Asterisk conseguia, por si só:

- registo SIP dos terminais
- autenticação de utilizadores
- chamadas internas
- comunicação com servidores de outros grupos

Assim, a introdução de um SIP Proxy externo não teria impacto positivo nos resultados esperados.

## 3. Kamailio é recomendado para infraestruturas massivas

O Kamailio só revela o seu potencial em:

- ambientes com milhares de utilizadores
- arquiteturas distribuídas
- requisitos de alta disponibilidade

Num laboratório académico com poucas extensões, esta abordagem é tecnicamente desadequada.

## 4. A sua presença introduziu instabilidade

Nos testes realizados, o Kamailio:

- ocupava a porta 5060
- interferia com o processo de registo dos telefones
- impossibilitava o correto funcionamento do Asterisk
- causava falhas nas chamadas internas

Para garantir um sistema VoIP estável e funcional, a desativação do Kamailio foi a opção tecnicamente correta.

---

# 7. Conclusão

O projeto permitiu implementar com sucesso uma infraestrutura VoIP funcional e estável, integrando DNS autoritativo, autenticação centralizada e um servidor Asterisk totalmente operacional. Os terminais SIP foram configurados com êxito, permitindo comunicações internas fiáveis e resoluções de nomes consistentes através do Bind9.

A tentativa de introduzir o Kamailio como proxy SIP, embora tecnicamente relevante, revelou-se desnecessária para os objetivos da unidade curricular e originou incompatibilidades com a versão disponível. A sua remoção foi uma decisão adequada, garantindo a estabilidade do sistema final.

No conjunto, o trabalho resultou numa solução VoIP coerente, segura e alinhada com práticas reais de redes e comunicações, demonstrando domínio das tecnologias envolvidas e capacidade de diagnóstico e decisão técnica.

---

CENTRO 2030 Os Fundos Europeus mais próximos de si. PORTUGAL 2030 Cofinanciado pela União Europeia